# Machine Learning Assignment – Unit 1 & Unit 2

---

## Q1. Disciplines and Their Influence on Machine Learning

| Discipline | Influence & Example |
|---|---|
| **Mathematics & Statistics** | Probability, linear algebra, calculus — e.g., Bayes' theorem in Naïve Bayes classifiers |
| **Neuroscience** | Brain architecture → Artificial Neural Networks, CNNs (visual cortex) |
| **Computer Science** | Algorithms & complexity → Decision Trees, efficient search |
| **Physics** | Statistical mechanics → Boltzmann Machines; Thermodynamics → Diffusion Models |
| **Linguistics** | Grammar theories → NLP; Transformers modeled on cognition |
| **Psychology** | Skinner's reinforcement → Reinforcement Learning (rewards & penalties) |
| **Information Theory** | Shannon's entropy → Cross-entropy loss, Decision Tree splitting (information gain) |
| **Economics/Game Theory** | Nash equilibria → GAN training (generator vs. discriminator) |
| **Biology** | Natural selection → Genetic Algorithms; Swarm intelligence → Particle Optimization |
| **Control Theory** | Kalman filters → Early RL; used extensively in robotics |

---

## Q2. Choices in Designing a Learning Example

Designing a learning example involves several key decisions:

### 1. Target Function

What exactly should the system learn? E.g., EnjoySport — learning whether a person enjoys a sport.

### 2. Training Experience

- **Direct vs. Indirect:** Direct feedback (labeled examples) vs. indirect (game outcomes in

chess).

- **Teacher vs. No Teacher:** Supervised (labeled data) vs. unsupervised (clustering).
- **Representativeness:** Training data must represent the real distribution.

### 3. Representation of Target Function

Choose how to represent the learned function:

- Decision Trees, Neural Networks, Linear functions, Rules, etc.

### 4. Learning Algorithm

Choose an algorithm suited to the representation: ID3, Backpropagation, Candidate Elimination, etc.

### 5. Performance Measure

How is success evaluated? Accuracy, F1-score, entropy minimization, etc.

**Example:** Checkers learning program (Samuel, 1959):

- Target function: Board position → Win/Lose probability
- Experience: Games played against itself (indirect, no teacher)
- Representation: Polynomial over board features
- Algorithm: Gradient descent / TD learning
- Measure: % games won

---

## Q3. Appropriate and Inappropriate Applications of ML

### ✅ Appropriate Applications

1. **Email Spam Detection** — Pattern recognition from large labeled datasets
2. **Medical Diagnosis** — Learning from historical patient records and outcomes
3. **Speech Recognition** — Complex acoustic patterns too hard to hand-code

### ❌ Inappropriate Applications

1. **Mathematical Theorem Proving** — Requires symbolic deduction, not pattern learning
2. **Legal Contract Drafting** — Requires precise rule-following and accountability
3. **Safety-Critical Real-Time Control (e.g., aircraft autopilot decisions)** — No tolerance for probabilistic errors; requires deterministic, verified logic

---

## Q4. Definitions with Examples

### (i) Inductive Learning Hypothesis

> **"Any hypothesis found to approximate the target function well over a sufficiently large set of training examples will also approximate it well over unobserved examples."**

The learner assumes that the best hypothesis consistent with training data will generalize to unseen data.

**Example:** In EnjoySport, if `<Sunny, Warm, Normal, Strong, Warm, Same>` → Yes consistently, the learner hypothesizes `<Sunny, Warm, ?, Strong, ?, Same>` will also yield Yes.

---

### (ii) General-to-Specific Ordering of Hypotheses

A hypothesis **$h_1$ is more general than $h_2$** if every instance satisfying $h_2$ also satisfies $h_1$, but not vice versa.

- Most general: `<?, ?, ?, ?, ?, ?>` (accepts everything)
- Most specific: `<∅, ∅, ∅, ∅, ∅, ∅>` (accepts nothing)

**Example:**

- $h_1$ = `<?, ?, ?, ?, ?, ?>` (more general)
- $h_2$ = `<Sunny, ?, ?, ?, ?, ?>` (more specific than $h_1$)
- $h_3$ = `<Sunny, Warm, ?, ?, ?, ?>` (more specific than $h_2$)

This defines a **partial order** over hypothesis space.

---

### (iii) General Boundary (G) and Specific Boundary (S)

The **Version Space** VS(H,D) can be represented by two boundary sets:

- **S (Specific Boundary):** The set of **maximally specific** hypotheses consistent with training data. These are the least general hypotheses that still classify all positive examples correctly.
- **G (General Boundary):** The set of **maximally general** hypotheses consistent with training data. These are the most general hypotheses that still exclude all negative examples.

Every hypothesis in the version space lies between S and G.

**Example (EnjoySport after 2 positive examples):**

- S = `{<Sunny, Warm, Normal, Strong, Warm, Same>}` → tightened to fit positives
- G = `{<?, ?, ?, ?, ?, ?>}` → still maximally general

---

### (iv) Version Space

The **Version Space** VS(H,D) with respect to hypothesis space H and training data D is the subset of hypotheses consistent with **all** training examples:

> VS(H,D) = { h ∈ H | h is consistent with every example in D }

**Example:** In EnjoySport, after seeing positive and negative examples, only those hypotheses that correctly classify all examples remain in the version space.

---

### (v) Version Space Representation Theorem

> **"Let G be the set of maximally general and S be the set of maximally specific hypotheses in H consistent with D. Then:** VS(H,D) = { h ∈ H | ∃s ∈ S, ∃g ∈ G such that $g \geqslant_m h \geqslant_m s$ }"

This means every hypothesis in the version space is **more general than some member of S** and **more specific than some member of G**. The version space is fully represented by just G and S, not all intermediate hypotheses.

---

## Q5. Algorithms

### (i) Find-S Algorithm – Finding a Maximally Specific Hypothesis

**Goal:** Find the most specific hypothesis consistent with all positive training examples.

**Steps:**

1. Initialize h to the most specific hypothesis: `<∅, ∅, ∅, ∅, ∅, ∅>`
2. For each **positive** training example d:
    - For each attribute constraint $a_i$ in h:
        - If $a_i$ is satisfied by d, do nothing
        - Else replace $a_i$ with the next more general constraint that covers d
3. Ignore all negative examples
4. Output h

**Limitations:** Ignores negative examples; assumes no noise; outputs only one hypothesis.

---

### (ii) List-Then-Eliminate Algorithm

**Goal:** Maintain an explicit list of all consistent hypotheses.

**Steps:**

1. Initialize **VersionSpace** = list of every hypothesis in H
2. For each training example `<x, c(x)>`:
   - Remove every hypothesis h from VersionSpace that is inconsistent with the example: $h(x) \neq c(x)$
3. Output the remaining VersionSpace

**Limitation:** Computationally infeasible for large H; requires enumerating all hypotheses.

---

### (iii) Candidate Elimination Learning Algorithm

**Goal:** Maintain G and S boundaries of the version space.

**Steps:**

1. Initialize:
   - G = { `<?, ?, ?, ?, ?, ?>` } (most general)
   - S = { `<∅, ∅, ∅, ∅, ∅, ∅>` } (most specific)
2. For each training example d:
   - **If d is Positive:**
     - Remove from G any hypothesis inconsistent with d
     - For each s in S inconsistent with d:
       - Remove s from S
       - Add all minimal generalizations h of s such that h is consistent with d and some member of G is more general than h
       - Remove from S any hypothesis more general than another in S
   - **If d is Negative:**
     - Remove from S any hypothesis inconsistent with d
     - For each g in G inconsistent with d:
       - Remove g from G
       - Add all minimal specializations h of g such that h is consistent with d and some member of S is more specific than h
       - Remove from G any hypothesis less general than another in G

3. Output G and S

---

## Q6. Candidate Elimination Trace – EnjoySport

**Attributes:** Sky, AirTemp, Humidity, Wind, Water, Forecast

**Initial:**

- G = { `<?, ?, ?, ?, ?, ?>` }
- S = { `<∅, ∅, ∅, ∅, ∅, ∅>` }

**Hypothesis H' to check:** `<?, Cold, High, ?, ?, ?>` ∨ `<Sunny, ?, High, ?, ?, Same>`

---

**After Example 1:** `<Sunny, Warm, Normal, Strong, Warm, Same>` → **Yes (+)**

- S → `<Sunny, Warm, Normal, Strong, Warm, Same>`
- G unchanged → `<?, ?, ?, ?, ?, ?>`

---

**After Example 2:** `<Sunny, Warm, High, Strong, Warm, Same>` → **Yes (+)**

- S: Generalize to cover both positives:
  - S → `<Sunny, Warm, ?, Strong, Warm, Same>`
- G unchanged

---

**After Example 3:** `<Rainy, Cold, High, Strong, Warm, Change>` → **No (−)**

- S is consistent (doesn't cover this negative) → unchanged
- G must be specialized to exclude this negative:
  - G generates specializations of `<?, ?, ?, ?, ?, ?>` that exclude `<Rainy, Cold, High, Strong, Warm, Change>`:
    - `<Sunny, ?, ?, ?, ?, ?>` ✓
    - `<?, Warm, ?, ?, ?, ?>` ✓
    - `<?, ?, Normal, ?, ?, ?>` ✓
    - `<?, ?, ?, ?, ?, Same>` ✓

- (others excluded for not being consistent with S)
- G = { `<Sunny,?,?,?,?,?>`, `<?,Warm,?,?,?,?>`, `<?,?,Normal,?,?,?>`, `<?,?,?,?,?,Same>` }

---

**After Example 4:** `<Sunny, Warm, High, Strong, Cold, Change>` **→ Yes (+)**

- S: Generalize to cover new positive:
  - S → `<Sunny, Warm, ?, Strong, ?, ?>`
- G: Remove members inconsistent with this positive:
  - `<?,?,Normal,?,?,?>` → inconsistent (Humidity=High ≠ Normal) → **remove**
  - `<?,?,?,?,?,Same>` → inconsistent (Forecast=Change ≠ Same) → **remove**
  - Remaining: G = { `<Sunny,?,?,?,?,?>`, `<?,Warm,?,?,?,?>` }

---

**Final Result:**

- **G = { `<Sunny,?,?,?,?,?>`, `<?,Warm,?,?,?,?>` }**
- **S = { `<Sunny, Warm, ?, Strong, ?, ?>` }**

**Checking H' against final VS:**

1. `<?, Cold, High, ?, ?, ?>` — Not consistent with S (AirTemp=Cold not covered). **Not in VS.**
2. `<Sunny, ?, High, ?, ?, Same>` — Not consistent with S (Forecast=Same excluded). **Not in VS.**

**∴ Neither disjunct of H' lies within the final version space.**

---

## Q7. Entropy and Information Gain

**Training Data:**

| Instance | Class | A1 | A2 |
|---|---|---|---|
| 1 | + | T | T |
| 2 | + | T | T |
| 3 | − | T | F |
| 4 | + | F | F |
| 5 | − | F | T |
| 6 | − | F | T |

Total: 3 positive (+), 3 negative (−), n = 6

---

**(i) Entropy of the Collection**

$$Entropy(S) = -p_+ \log_2 p_+ - p_- \log_2 p_-$$

- $p_+ = 3/6 = 0.5$, $p_- = 3/6 = 0.5$

$$Entropy(S) = -0.5 \log_2(0.5) - 0.5 \log_2(0.5) = -0.5(-1) - 0.5(-1) = \mathbf{1.0} \text{ bit}$$

---

**(ii) Information Gain of A2**

**A2 = T:** Instances 1, 2, 5, 6 → +,+,−,− → $p_+=2/4$, $p_-=2/4$

$$Entropy(S_{A2=T}) = -0.5 \log_2(0.5) - 0.5 \log_2(0.5) = 1.0$$

**A2 = F:** Instances 3, 4 → −,+ → $p_+=1/2$, $p_-=1/2$

$$Entropy(S_{A2=F}) = -0.5 \log_2(0.5) - 0.5 \log_2(0.5) = 1.0$$

$$Gain(S, A2) = Entropy(S) - \frac{|S_T|}{|S|} \cdot Entropy(S_T) - \frac{|S_F|}{|S|} \cdot Entropy(S_F)$$

$$= 1.0 - \frac{4}{6}(1.0) - \frac{2}{6}(1.0) = 1.0 - 0.667 - 0.333 = \mathbf{0.0} \text{ bits}$$

**A2 provides zero information gain** — it does not help discriminate the classes.

---

## Q8. Definitions

### (i) Maximum Likelihood Estimation (MLE)

MLE finds the parameter θ that **maximizes the likelihood** of observing the training data D:

$$\hat{\theta}_{MLE} = \arg\max_{\theta} P(D|\theta)$$

It selects the hypothesis that makes the observed data most probable. It does **not** consider any prior belief about θ.

**Example:** Estimating the bias of a coin — MLE gives θ = (number of heads)/(total flips).

---

### (ii) Maximum A Posteriori (MAP) Estimation

MAP finds the parameter θ that maximizes the **posterior probability**, incorporating a prior:

$$\hat{\theta}_{MAP} = \arg\max_{\theta} P(\theta|D) = \arg\max_{\theta} P(D|\theta) \cdot P(\theta)$$

It combines the likelihood with a **prior distribution** P(θ), making it less prone to overfitting on small datasets.

**Example:** If we believe the coin is likely fair (strong prior around θ=0.5), MAP will pull the estimate toward 0.5, even if a few early flips are all heads.

---

## Q9. MLE – Statement and Proof

### Statement

Given training data D = $\{d_1, d_2, ..., d_n\}$ generated i.i.d. from distribution P(D|θ), the MLE estimate is:

$$\hat{\theta}_{MLE} = \arg\max_{\theta} P(D|\theta) = \arg\max_{\theta} \prod_{i=1}^{n} P(d_i|\theta)$$

**Proof / Derivation**

Since the product is hard to maximize directly, we take the log (monotonic transformation):

$$\hat{\theta}_{MLE} = \arg\max_{\theta} \ln P(D|\theta) = \arg\max_{\theta} \sum_{i=1}^{n} \ln P(d_i|\theta)$$

**For Bernoulli example** (coin with bias θ): Let $n_1$ = heads, $n_0$ = tails, $n = n_1 + n_0$

$$\ln L(\theta) = n_1 \ln \theta + n_0 \ln(1-\theta)$$

Differentiate and set to zero:

$$\frac{d}{d\theta} \ln L = \frac{n_1}{\theta} - \frac{n_0}{1-\theta} = 0$$

$$n_1(1-\theta) = n_0\theta \implies n_1 = \theta(n_1 + n_0)$$

$$\boxed{\hat{\theta}_{MLE} = \frac{n_1}{n_1 + n_0}}$$

This confirms MLE = empirical frequency. ■

---

## Q10. MAP – Statement and Proof

**Statement**

Given prior P(θ) and likelihood P(D|θ), by Bayes' theorem:

$$P(\theta|D) = \frac{P(D|\theta) \cdot P(\theta)}{P(D)}$$

Since P(D) is constant with respect to θ:

$$\hat{\theta}_{MAP} = \arg\max_{\theta} P(D|\theta) \cdot P(\theta)$$

**Proof / Derivation**

Taking the log:

$$\hat{\theta}_{MAP} = \arg\max_{\theta} \left[\ln P(D|\theta) + \ln P(\theta)\right]$$

**For Bernoulli with Beta prior** θ ~ Beta(α, β):

$$\ln P(\theta) = (\alpha - 1)\ln\theta + (\beta - 1)\ln(1 - \theta) + \text{const}$$

$$\ln P(D|\theta) + \ln P(\theta) = (n_1 + \alpha - 1)\ln\theta + (n_0 + \beta - 1)\ln(1 - \theta)$$

Differentiate and set to zero:

$$\frac{n_1 + \alpha - 1}{\theta} - \frac{n_0 + \beta - 1}{1 - \theta} = 0$$

$$\boxed{\hat{\theta}_{MAP} = \frac{n_1 + \alpha - 1}{n_1 + n_0 + \alpha + \beta - 2}}$$

When α = β = 1 (uniform prior), MAP = MLE. As n → ∞, the prior becomes negligible and MAP → MLE. ■

---

## Q11. Decision Trees for Boolean Functions

### (i) A ∧ ¬B (A AND NOT B)

```
        A
       / \
    F     T
    |     |
   No     B
         / \
       T    F
       |    |
      No   Yes
```

**Truth Table:**

| A | B | A ∧ ¬B |
|---|---|---|
| F | F | F |
| F | T | F |
| T | F | **T** |
| T | T | F |

Only True when A=T and B=F.

---

### (ii) A ∨ (B ∧ C)

```
          A
         / \
        T   F
        |    \
       Yes    B
             / \
            F   T
            |    \
           No     C
                 / \
                F   T
                |   |
               No  Yes
```

**Truth Table:**

| **A** | **B** | **C** | **A∨(B∧C)** |
|---|---|---|---|
| F | F | F | F |
| F | F | T | F |
| F | T | F | F |
| F | T | T | **T** |
| T | F | F | **T** |
| T | F | T | **T** |
| T | T | F | **T** |
| T | T | T | **T** |

True when A=T, OR when A=F and both B=T and C=T.